NNGraph DSL Compiler

A Domain-Specific Language for Describing Neural Network Architectures and Automatic PyTorch Code Generation

Project Technical Report

June 29, 2026

Contents

Abstract

NNGraph DSL is a domain-specific language designed to describe neural network architectures as directed graphs. It accepts `.nng` files as input and produces executable PyTorch code consisting of a complete `nn.Module` subclass with `__init__` and `forward` methods.

The compiler is built with ANTLR4 version 4.13.2 and uses the Listener pattern for semantic analysis. Kahn's topological sort algorithm is employed both in the semantic analyzer (cycle detection) and in the code generator (execution ordering). Tensor shape inference, Graphviz graph visualization, training scaffolding, quantization annotations, and ONNX export are also supported.

The test suite comprises 55 tests across four files. Four example architectures—MLP, ResBlock, InceptionBlock, and TransformerEncoder—compile successfully and produce correct output tensor shapes in their forward pass.

1 Introduction

1.1 Motivation

Writing `nn.Module` subclasses in PyTorch by hand for complex architectures is repetitive and error-prone. For a network with ten layers, the programmer must:

- Define each layer separately with correct parameters in `__init__`
- Manually manage the execution order in `forward`
- Name intermediate variables in branching topologies
- Ensure no shared tensor is overwritten before it is consumed

Consider the following PyTorch code that must be written manually:

Listing 1.1.1 ▸ Manual boilerplate for a ResBlock

```python
class ResBlock(nn.Module):
    def __init__(self):
        super(ResBlock, self).__init__()
        self.conv1 = nn.Conv2d(
            in_channels=64, out_channels=64,
            kernel_size=3, stride=1, padding=1)
        self.bn1   = nn.BatchNorm2d(num_features=64)
        self.relu1 = nn.ReLU()
        # ... and so on for each layer

    def forward(self, x):
        identity = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu1(x)
        # ... manage residual connection manually
```

```
        x = x + identity
        return x
```

With NNGraph DSL, the architecture is described once as a graph and the compiler generates this code automatically.

1.2 Goal

The goal of NNGraph DSL is to make the `.nng` file the single source of truth for the architecture. The programmer describes the graph; the compiler generates all boilerplate.

1.3 Scope

This tool is suited for describing static architectures—networks whose computational graph is known at model definition time. Support for dynamic control flow within `forward` is planned for future versions.

2

Related Work

2.1 The NADER Framework

Definition 2.1.1 ▸ NADER

NADER (Neural Architecture Design via Representation) is a framework that models neural network architectures as directed computational graphs (DAGs). Each node represents a layer operation and each edge represents tensor flow.

NNGraph DSL adopts this graph model as the foundation of its language design.

2.2 ANTLR4

Definition 2.2.1 ▸ ANTLR4

ANTLR4 (ANother Tool for Language Recognition) is a tool that automatically generates a lexer, parser, and listener/visitor from a `.g4` grammar file. Version used: ANTLR 4.13.2.

2.2.1 Listener Pattern vs. Visitor Pattern

ANTLR4 provides two patterns for traversing the parse tree:

- **Listener**: `ParseTreeWalker` automatically traverses the tree and calls `enter*/exit*` methods. No return values are needed.
- **Visitor**: The programmer controls traversal and each method returns a value.

> This compiler uses the Listener pattern. The `exit*` methods in `custom_listener.py` accumulate state into `self.nodes` and `self.edges`—this approach is more natural for building a graph model from a parse tree.

2.3 EVM ANTLR Project

This compiler inherits its code style and structure from an existing EVM project in `PycharmProjects/Ar`
Three files from that project are used directly:

- `ast.py` — `AST` and `TreeNode` classes
- `make_ast_subtree.py` — parse tree node composition
- `ast_to_networkx_graph.py` — optional graph visualization

3 Language Design

3.1 Program Structure

Every `.nng` file consists of three blocks. The `config` block is optional:

Listing 3.1.1 ▸ General structure of a .nng file

```
model <Name> {
    input  <id> : tensor(<shape>)
    output <id>
}

graph {
    node <id> : <LayerType>(<params>)
    ...
    edge <src> -> <dst>
    edge <src> -> <dst> [label="<string>"]
    ...
}

config {
    batch_size = <int>
    device     = "cpu" | "cuda"
    loss       = "<LossFn>"       // enables training scaffold
    optimizer  = "<Optimizer>"    // Adam | SGD | AdamW | RMSprop
    lr         = <float>          // learning rate
    epochs     = <int>            // training epochs
}
```

Definition 3.1.2 ▸ Model block

The `model` block specifies the generated class name, the input node identifier and tensor shape, and the output node identifier.

Definition 3.1.3 ▸ Graph block

The `graph` block defines all layer nodes and directed edges. Edge declaration order does not matter; the compiler computes execution order from the graph structure.

Definition 3.1.4 ▸ Config block

The `config` block sets values for the `__main__` block of the generated code. `batch_size` defaults to 1 and `device` defaults to `'cpu'`. If the `loss` key is set, the code generator produces a complete training loop with optimizer and loss function.

3.2 Grammar

Definition 3.2.1 ▸ NNGraph.g4 statistics

- Grammar file: `NNGraph.g4` (56 lines)
- Parser rules: 14 rules (snake_case)
- Lexer tokens: 20 tokens (ALL_CAPS)

Complete grammar rules:

Listing 3.2.2 ▸ NNGraph.g4 grammar

```
grammar NNGraph;
program      : model_block graph_block config_block? EOF;
model_block  : MODEL ID '{' input_decl output_decl '}';
input_decl   : INPUT ID ':' TENSOR shape_expr;
output_decl  : OUTPUT ID;
graph_block  : GRAPH '{' (node_decl | edge_decl)* '}';
node_decl    : NODE ID ':' layer_expr;
layer_expr   : ID '(' param_list? ')';
param_list   : param (',' param)*;
param        : ID '=' value;
```

```
edge_decl   : EDGE ID ARROW ID ('[' LABEL '=' STRING ']')?;
config_block: CONFIG '{' config_entry* '}';
config_entry: ID '=' value;
value       : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
            | STRING | NONE | shape_expr;
shape_expr  : '(' INT_LITERAL (',' INT_LITERAL)* ')';
```

Important design note: `input_decl` uses `TENSOR shape_expr` rather than `TENSOR '(' shape_expr ')'` because `shape_expr` itself includes the parentheses. This common mistake in early grammar versions was corrected.

3.3 Type System

DSL Type	Python Type	Example
`int`	`int`	`128`
`float`	`float`	`0.1`
`bool`	`bool`	`true` / `false`
`string`	`str`	`"relu"`
`shape`	`tuple[int]`	`(64, 32, 32)`
`None`	`NoneType`	`None`

The distinction between `int` and `float` is enforced at compile time. For example, `Dropout(p=1)` produces an error because `p` expects a `float`, not an `int`.

3.4 Layer Types

3.4.1 Parameterized Layers

These layers generate a `self.<id> = nn.X(...)` line in `__init__`.

DSL Keyword	PyTorch Class	Key Parameters
`Linear`	`nn.Linear`	`in_features`, `out_features`, `bias`
`Conv2d`	`nn.Conv2d`	`in_ch`, `out_ch`, `kernel`, `stride`, `padding`
`Conv1d`	`nn.Conv1d`	`in_ch`, `out_ch`, `kernel`, `stride`
`BatchNorm2d`	`nn.BatchNorm2d`	`num_features`
`LayerNorm`	`nn.LayerNorm`	`normalized_shape`
`MaxPool2d`	`nn.MaxPool2d`	`kernel`, `stride`
`AvgPool2d`	`nn.AvgPool2d`	`kernel`, `stride`
`Dropout`	`nn.Dropout`	`p`
`Flatten`	`nn.Flatten`	`start_dim`, `end_dim`
`Embedding`	`nn.Embedding`	`num_embeddings`, `embedding_dim`
`MultiHeadAttn`	`nn.MultiheadAttention`	`embed_dim`, `num_heads`
`LSTM`	`nn.LSTM`	`input_size`, `hidden_size`, `num_layers`
`GRU`	`nn.GRU`	`input_size`, `hidden_size`

3.4.2 Activations

`ReLU`, `Sigmoid`, `Tanh`, `GELU`, `Softmax(dim)`, `LeakyReLU(negative_slope)`, `ELU(alpha)`.

3.4.3 Graph Operations (No Module)

These types do not generate a line in `__init__`:

Keyword	Parameters	Generated Code
`Add`	—	`out = a + b`
`Concat`	`dim: int`	`out = torch.cat([a,b], dim=d)`
`Residual`	—	`out = main + shortcut`
`Split`	`chunks: int, dim: int`	`split_0, split_1, ... = torch.chunk(x,n,di`

3.4.4 Universal Parameters

The `quant` parameter can be applied to any node regardless of layer type. Valid values: `"dynamic"`, `"static"`, `"qat"`.

Listing 3.4.1 ▸ Quantization annotation example

```
node fc1 : Linear(in_features=784, out_features=128,
↪   quant="dynamic")
```

3.5 Edge Syntax

Definition 3.5.1 ▸ Simple edge

`edge src -> dst`

Defines a directed edge from node `src` to node `dst`.

Definition 3.5.2 ▸ Labeled edge

`edge src -> dst [label="name"]`

Defines an edge with a string label. Labels serve two roles:

- `"shortcut"`: marks the skip connection edge into a `Residual()` node.
- `"residual_*"`: marks a skip connection into a node with two inputs (Transformer pattern).

3.6 Complete Example: MLP

The `.nng` file for a three-layer network:

Listing 3.6.1 ▸ mlp.nng

```
model MLP {
    input  x   : tensor(784)
    output out
}

graph {
```

```
    node fc1   : Linear(in_features=784, out_features=256)
    node relu1 : ReLU()
    node fc2   : Linear(in_features=256, out_features=128)
    node relu2 : ReLU()
    node drop  : Dropout(p=0.3)
    node fc3   : Linear(in_features=128, out_features=10)
    node out   : Softmax(dim=1)

    edge x     -> fc1
    edge fc1   -> relu1
    edge relu1 -> fc2
    edge fc2   -> relu2
    edge relu2 -> drop
    edge drop  -> fc3
    edge fc3   -> out
}

config {
    batch_size = 64
    device = "cuda"
}
```

Generated PyTorch code:

Listing 3.6.2 ▸ Generated PyTorch code for MLP

```python
import torch
import torch.nn as nn


class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1   = nn.Linear(in_features=784,
         ↪  out_features=256)
        self.relu1 = nn.ReLU()
```

```python
        self.fc2   = nn.Linear(in_features=256, out_features=128)
        self.relu2 = nn.ReLU()
        self.drop  = nn.Dropout(p=0.3)
        self.fc3   = nn.Linear(in_features=128, out_features=10)
        self.out   = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.drop(x)
        x = self.fc3(x)
        x = self.out(x)
        return x


if __name__ == '__main__':
    device = torch.device('cuda')
    model  = MLP().to(device)
    x      = torch.randn(64, 784).to(device)
    print(model(x).shape)
```

4 Implementation

4.1 Compiler Architecture

Definition 4.1.1 ‣ Compilation pipeline

The compiler has six stages:

1. **Parsing**: `FileStream` → `NNGraphLexer` → `NNGraphParser.program()` → parse tree
2. **Semantic analysis**: `ParseTreeWalker` + `NNGraphCustomListener` → graph model
3. **Topological sort**: compute execution order using Kahn's algorithm
4. **Shape inference**: propagate tensor shapes through the graph and detect dimension mismatches (optional: `--show-shapes`)
5. **Code generation**: `CodeGenerator.generate()` → Python string
6. **Output**: write to `.py` file, Graphviz DOT output, or ONNX export

4.1.1 File Responsibilities

File	Role
`NNGraph.g4`	Grammar source of truth (56 lines)
`gen/NNGraphLexer.py`	Generated by ANTLR4
`gen/NNGraphParser.py`	Generated by ANTLR4
`gen/NNGraphListener.py`	Base listener — should not be edited
`custom_listener.py`	Semantic analysis
`code_generator.py`	PyTorch code generation
`shape_inference.py`	Compile-time tensor shape inference
`graph_viz.py`	Graphviz DOT visualization
`onnx_export.py`	ONNX model export
`main.py`	CLI interface and `compile_nng()`

4.2 Semantic Analyzer

Definition 4.2.1 ▸ NNGraphCustomListener

The class `NNGraphCustomListener(NNGraphListener)` in `custom_listener.py` implements all semantic analysis logic. Internal state:

- `self.nodes`: `dict[id, {type, params, line}]`
- `self.edges`: `list[{src, dst, label, line}]`
- `self.config`: `dict[key, value]`

4.2.1 Value Propagation in the Tree

The `exitShape_expr` and `exitValue` methods store values on the context object itself:

Listing 4.2.2 ▸ Value propagation via ctx.pvalue in exitShape_expr

```
def exitShape_expr(self, ctx):
    ints = [int(ctx.INT_LITERAL(i).getText())
            for i in range(ctx.INT_LITERAL().__len__())]
```

```
    ctx.pvalue = tuple(ints)
    ctx.ptype  = tuple
```

This approach ensures that each context, after exiting, has its children's values available—a pattern also used in the reference EVM project.

4.2.2 Validation Rules

Eight validation rules are executed in two phases:

Phase one (during tree walk):

1. **Duplicate node ID**: in `exitNode_decl`
2. **Invalid edge reference**: in `exitEdge_decl`
3. **Unknown layer type**: in `exitNode_decl`
4. **Parameter type mismatch**: in `exitNode_decl`

Phase two (in `exitGraph_block`, after graph completion):

5. **Undeclared output node**: in `_validate_output_declared`
6. **Orphan node**: in `_validate_orphans`
7. **Residual arity $\neq$ 2**: in `_validate_residual_arity`
8. **Cycle detection via Kahn's algorithm**: in `_validate_no_cycles`
9. **BFS reachability**: in `_validate_reachability`

4.2.3 Error Message Format

Listing 4.2.3 ▸ Error format

```
[line L:C] SemanticError: <message>
[line L] ShapeError at '<node>': <message>
```

`L` is the line number (1-based) and `C` is the token column (0-based). All error conditions halt compilation and exit with code 1.

4.2.4 _FakeCtx for Post-Walk Errors

Errors raised in `exitGraph_block` do not have a live context. An internal class `_FakeCtx` carries the line number from the node's declaration record:

Listing 4.2.4 ▸ _FakeCtx class for preserving line numbers

```
class _FakeCtx:
    class start:
        line   = info["line"]
        column = 0
```

4.3 Code Generator

Definition 4.3.1 ▸ CodeGenerator

The `CodeGenerator` class in `code_generator.py` has four internal steps:

1. `_compute_graph()`: build adjacency lists and topological sort
2. `_assign_vars()`: assign a Python variable name to each node
3. `_emit_init()` + `_emit_forward()`: generate code lines
4. `_assemble()`: combine into final Python source

4.4 Variable Assignment Algorithm

Definition 4.4.1 ▸ Core variable assignment invariant

`var_at[n] = 'x'` only when `out_degree[predecessor] == 1`.
When a node fans out to multiple destinations (`out_degree > 1`), its tensor is still needed by other branches. Overwriting `x` at this point would corrupt it. Instead, the downstream node receives its own identifier as a variable name.

The four complete rules in `_assign_vars`:

Listing 4.4.2 ▸ Variable assignment rules in __assign__vars

```
Rule 1 - Input node:
    var_at[input_id] = 'x'
```

```

Rule 2 - Merge node (in_degree > 1):
    var_at[n] = 'x'        if out_degree[n] <= 1
    var_at[n] = node_id    if out_degree[n] > 1

Rule 3 - Branch start (predecessor.out_degree > 1):
    var_at[n] = node_id

Rule 4 - Linear continuation (predecessor.out_degree == 1):
    var_at[n] = var_at[predecessor]
```

4.4.1 MLP Trace (Linear Chain)

Example 4.4.3 ▸ Variable trace for MLP

Topology: `x -> fc1 -> relu1 -> fc2 -> relu2 -> drop -> fc3 -> out`

Every predecessor has `out_degree == 1` ⇒ Rule 4 always applies ⇒ all nodes have `var_at = 'x'`. The `forward` method uses `x` everywhere.

4.4.2 ResBlock Trace (Fan-Out Then Merge)

Example 4.4.4 ▸ Variable trace for ResBlock

`x` fans out to `conv1` and `skip` (shortcut). Therefore `out_degree[x] = 2`.

- `conv1`: Rule 3 ⇒ `var_at = 'conv1'`
- `bn1, relu1, conv2, bn2`: Rule 4 ⇒ all `'conv1'`
- `skip` (Residual, `in_degree=2`): Rule 2, `out_degree=1` ⇒ `var_at = 'x'`

Generated code:

Listing 4.4.5 ▸ Generated forward pass for ResBlock

```
conv1 = self.conv1(x)
conv1 = self.bn1(conv1)
conv1 = self.relu1(conv1)
conv1 = self.conv2(conv1)
conv1 = self.bn2(conv1)
x     = conv1 + x        # Residual node
x     = self.flatten(x)
```

4.4.3 InceptionBlock Trace (Two Parallel Branches)

Example 4.4.6 ▸ Variable trace for InceptionBlock

`out_degree[x] = 2` (to `convA1` and `convB1`).

- `convA1`: Rule 3 ⇒ `'convA1'`
- `convB1`: Rule 3 ⇒ `'convB1'`
- `convA2`: Rule 4 ⇒ inherits `'convA1'`
- `convB2`: Rule 4 ⇒ inherits `'convB1'`
- `cat1` (Concat, `in_degree=2`): Rule 2 ⇒ `'x'`

Generated code:

Listing 4.4.7 ▸ Generated forward pass for InceptionBlock

```
convA1 = self.convA1(x)
convA1 = self.convA2(convA1)
convB1 = self.convB1(x)
convB1 = self.convB2(convB1)
x      = torch.cat([convA1, convB1], dim=1)  # Concat node
x      = self.bn(x)
x      = self.out(x)
```

4.4.4 TransformerEncoder Trace (Two Implicit Residuals)

Example 4.4.8 ▸ Variable trace for TransformerEncoder

`out_degree[x] = 2` and `out_degree[norm1] = 2`.

- `attn`: Rule 3 ⇒ `'attn'`
- `drop1`: Rule 4 ⇒ `'attn'`
- `norm1` (`in_deg=2, out_deg=2`): Rule 2 ⇒ `'norm1'`
- `ff1`: Rule 3 ⇒ `'ff1'`
- `gelu1, drop2, ff2`: Rule 4 ⇒ `'ff1'`
- `norm2` (`in_deg=2, out_deg=1`): Rule 2 ⇒ `'x'`

Generated code:

Listing 4.4.9 ▸ Generated forward pass for TransformerEncoder

```python
attn, _ = self.attn(x, x, x)    # MultiHeadAttn
    triple-pass
attn      = self.drop1(attn)
norm1     = self.norm1(x + attn)  # implicit residual_1
ff1       = self.ff1(norm1)
ff1       = self.gelu1(ff1)
ff1       = self.drop2(ff1)
ff1       = self.ff2(ff1)
x         = self.norm2(norm1 + ff1) # implicit
    residual_2
x         = self.out(x)
x         = self.flatten(x)
return x
```

4.4.5 Special Code Generation Cases

Pattern	Condition	Generated Code
MultiHeadAttn	Layer type	`out, _ = self.attn(x, x, x)`
LSTM / GRU	Layer type	`out, _ = self.lstm(x)`
Residual	No module	`out = main + shortcut`
Add	No module	`out = a + b`
Concat	No module	`out = torch.cat([a,b], dim=d)`
Split	No module	`split_0, split_1, ... = torch.chunk(x,n,dim=d)`
Implicit residual	`residual_*` label	`out = self.norm(skip + main)`

Parameter name mapping: The DSL uses shorter names. The code generator maps them to PyTorch names: `in_ch` → `in_channels`, `out_ch` → `out_channels`, `kernel` → `kernel_size`.

4.4.6 Output Code Structure

Listing 4.4.10 ▸ Generated output code template

```python
import torch
import torch.nn as nn


class <ModelName>(nn.Module):
    def __init__(self):
        super(<ModelName>, self).__init__()
        # one line per parameterized layer node

    def forward(self, x):
        # one line per node in topological order
        return <output_var>


if __name__ == '__main__':
    device = torch.device('<device>')
```

```
    model  = <ModelName>().to(device)
    x      = torch.randn(<batch_size>, <input_shape>).to(device)
    print(model(x).shape)
```

4.5 Shape Inference

Definition 4.5.1 ‣ Shape Inference

The `shape_inference.py` module propagates tensor shapes from the input node along the topological order. For each layer type, specific rules are applied: Linear changes the last dimension, Conv2d computes spatial dimensions, Flatten merges dimensions, and so on.

When a dimension mismatch is detected, a `ShapeError` is reported with the line number of the node declaration, and compilation is halted. The `--show-shapes` flag displays each node's output shape.

4.6 Graphviz Visualization

The `graph_viz.py` module produces Graphviz DOT output. Nodes are color-coded by layer type (green for input, blue for Linear, orange for Conv, purple for Normalization). The output node is marked with `peripheries=2` and labeled edges are shown with dashed lines. If shape inference is enabled, shapes are annotated on the nodes.

Listing 4.6.1 ‣ Generating and rendering a graph

```
python main.py -i input/inception.nng --graph-viz output/inception.dot
dot -Tpng output/inception.dot -o output/inception.png
```

4.7 Training Scaffold

If the `loss` key is set in the `config` block, the code generator produces a complete training loop including loss function, optimizer, and epoch loop. Supported loss functions:

`CrossEntropyLoss`, `MSELoss`, `BCELoss`, `BCEWithLogitsLoss`, `L1Loss`, `NLLLoss`, `SmoothL1Loss`. Supported optimizers: `Adam`, `SGD`, `AdamW`, `RMSprop`.

4.8 Quantization Annotations

The universal `quant` parameter can be applied to any node. Three modes are supported: `"dynamic"`, `"static"`, and `"qat"`. QAT nodes are wrapped in `torch.quantization.QuantWrapp` When at least one node has a quantization annotation, `QuantStub` and `DeQuantStub` are added to `__init__` and `forward`.

4.9 ONNX Export

The `onnx_export.py` module instantiates the generated code and uses `torch.onnx.export` to produce an `.onnx` file with a dynamic batch axis and opset 17.

Listing 4.9.1 ▸ Exporting a model to ONNX

```
python main.py -i input/mlp.nng --onnx output/mlp.onnx
```

5 Evaluation

5.1 Test Suite

Definition 5.1.1 ▸ Test statistics

- Total tests: **55**
- Framework: `pytest` version 8.4.2
- Runtime: approximately 1.09 seconds

File	Tests	Coverage
`test_parser.py`	8	Grammar parses all 4 examples with zero syntax errors; config, label, shape
`test_listener.py`	14	4 valid inputs; 10 error conditions with `SystemExit`
`test_codegen.py`	19	nn class names, parameter values, forward patterns
`test_e2e.py`	14	Full compile + instantiate + forward pass with correct shape

Test execution command:

Listing 5.1.2 ▸ Running the test suite

```
python -m pytest tests/ -v
```

5.2 Forward Pass Results

Example 5.2.1 ▸ MLP

- Input: `tensor(784)` — Output: `(batch, 10)`
- Test: Softmax rows sum to 1 (`atol=1e-5`)

Example 5.2.2 ▸ ResBlock

- Input: `tensor(64, 32, 32)` — Output: `(batch, 65536)`
- Test: correct shape; deterministic output in `eval` mode

Example 5.2.3 ▸ InceptionBlock

- Input: `tensor(32, 56, 56)` — Output: `(batch, 48, 56, 56)`
- Branch A: 16 channels; Branch B: 32 channels; merged via `Concat(dim=1)` → 48 channels

Example 5.2.4 ▸ TransformerEncoder

- Input: `tensor(512, 128)` — Output: `(1024, 128)`
- Two implicit residual sub-layers without explicit `Residual()` nodes
- `MultiHeadAttn` with `embed_dim=128, num_heads=8, batch_first=True`

5.3 Variable Correctness Proof

Definition 5.3.1 ▸ Tensor non-overwrite proof

For each branching example, the input tensor's `data_ptr()` was traced through the forward computation:

- **ResBlock**: `x.data_ptr()` unchanged at the point `x = conv1 + x`
- **Inception**: `x.data_ptr()` identical when `convA1(x)` and `convB1(x)` are called
- **Transformer**: `x.data_ptr()` unchanged when `norm1(x + attn)` is called; `norm1.data_ptr()` unchanged when `norm2(norm1 + ff1)` is called

This follows directly from the invariant: `var_at[n]='x'` only when

```
out_degree=1.
```

5.4 Codebase Statistics

File/Directory	**Lines**
`NNGraph.g4`	56
`custom_listener.py`	303
`code_generator.py`	327
`shape_inference.py`	193
`graph_viz.py`	90
`onnx_export.py`	47
`main.py`	121
`tests/` (4 files)	575
`required_code_collection/`	≈100
Total	≈1812

- Language: Python 3.14.3
- ANTLR: 4.13.2
- PyTorch: 2.10.0
- pytest: 8.4.2

6 Design Decisions

6.1 Listener vs. Visitor

The Listener pattern naturally suits state accumulation. The `exit*` methods can operate directly on `self.nodes` and `self.edges` without needing to return values through the tree.

6.2 load_from_listener() vs. generate(traversal)

The reference EVM project uses `CodeGenerator.generate(traversal)`, which receives a post-order token list—suited for stack-based bytecode generation. For a graph compiler, the natural input is graph structure (nodes + edges) rather than a linear token stream. Therefore, `load_from_listener(listener)` is more direct.

6.3 Variable Naming with node_id

The language specification examples use single-letter or descriptive names. This compiler uses the actual node identifier because:

- It is deterministic and works for larger graphs without a counter
- It is readable—the variable `convA1` is clearer than `a`
- The mathematical output is identical

7 Conclusion and Future Work

7.1 Conclusion

NNGraph DSL provides a six-stage compiler pipeline from `.nng` to executable `nn.Module`. The compiler features an ANTLR4 grammar with 14 parser rules, 24 layer types, 8 validation rules, and a variable assignment algorithm that handles complex branching without tensor overwrite. The compiler has been validated with 55 tests across four example architectures.

7.2 Implemented Features

Feature	Description
Shape inference	Tensor shape propagation and mismatch detection at compile time (`--show-s`
Graphviz visualization	Model graph output in DOT format (`--graph-viz`)
Training scaffold	Training loop generation with optimizer and loss from `config` block
Quantization	`quant` metadata for model quantization conversion
ONNX export	Model export to ONNX format (`--onnx`)

7.3 Future Work

Feature	Description
Named sub-graphs	Reusable macro blocks
Dynamic control flow	`if`/`else` branches within `forward`

References

1. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
2. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.
3. NADER Framework: Neural Architecture Design via Representation (referenced in `NNGraph_DSL_Specification.md`).
4. Zhang, A. M. *amznotes LaTeX template*. `github.com/alexmingzhang/latex-notes-templa`